IoT Predictive Maintenance Node using Multicore Edge AI

Piyush Maru(23BEC146) and Tejas Raichura(23BEC154)
Department of Electronics and Communication Engineering
Nirma University
Ahmedabad, India

***Abstract*—Predictive maintenance is an important part of industry, which ensures safety of industrial devices. In this paper, an industrial predictive maintenance implementation is carried out using ESP32 and online data training testing application called Edge AI. For getting the vibrations from the 12V Motor, MPU6050 accelerometer plus gyrometer vibration sensor is used.The singals are then processed by Edge Impulse AI model which is already dumped in the ESP32,to give probabilities of four operation modes. On the basis of probabilities obtained and priorities set within code, a message is displayed whether motor is in Idle mode, Normal Operation mode, Shaft Fault and Mounting Fault.**

***Index Terms*—Free RTOS, IoT, Predictive Maintenance, Edge AI, ESP32, MPU6050, Edge Impulse, Fault Detection**

I. Introduction

Every system needs constant monitoring to prevent sudden failures. Actually the methods which are used today cause increased downtime, which means that to repair them lot of time is wasted, also the repair costs increase. Predictive maintenance detects faults before they occur, thus giving the user an indication that something is wrong in the system. In this study, a smart monitoring device is built using an ESP32 microcontroller, an MPU6050 vibration sensor, and a 12V DC motor as the main device which is to be tested. The ESP32 runs on FreeRTOS, which acts like a task manager, which handles sensor reading, fault detection, and wireless data transmission without any interference. This makes the system fast, reliable, and capable of catching faults in real time.

II. Literature Survey

A. Cloud-Based Predictive Maintenance

Many IoT systems transmit sensor data to cloud platforms for fault detection using machine learning [1], [2]. Although this is accurate, it increases the time delay by a big factor. To solve this, systems like the proposed node use FreeRTOS-based edge processing, where data is actually worked upon in real time.

B. Edge-Based Fault Detection

Edge Computation is a new concept which is adopted for cloud latency problem solving. [3], [4]. The machine learning part and decision making part are directly done by the ESP32, thus decreasing time delay, reducing the usage of network, and enabling faster real-time monitoring in industrial environments.

C. Machine Learning for Motor Fault Detection

Machine learning is widely used for identifying faults in electric motors using vibration and different signals [5], [6]. A more advanced version is the Deep Learning model, which uses concept of convolutional neural networks to identify shaft and mounting fault. [7], [8]. We can also use the TinyML model, which is basically built for the low power devices. [9], [10].

D. Limitations of Existing Technology

The current systems used are more dependent on Cloud Systems for data processing, in which user has to wait for proper internet connectivity stabilization. This not only increases response time, but also the costs making it completely useless for small-scale time critical applications. Moreover, the methods consume a lot of power, so these are not really preferred.

III. Proposed Methodology

The proposed Edge AI predictive maintenance system is designed, modelled, and embedded as explained in this section. The design entails the acquisition of high-frequency raw data, on-device digital signal processing (DSP), quantised machine learning (Tiny ML) inference, and real-time asynchronous cloud telemetry in a Robust Dual-Core RTOS environment.

A. Hardware Architecture and Microcontroller Specifications

The proposed system's processor is an ESP32, a Dual-Core 32-bit LX6 microprocessor utilising the Xtensa architecture. Its clock frequency is 240 MHz, and it has a 520 KB internal SRAM. This microcontroller was chosen because it can perform complex tensor computations locally and already supports the 802.11 b/g/n Wi-Fi protocols for IoT connectivity. The kinematic data will be acquired with an MPU6050 Micro-Electro-Mechanical Systems (MEMS) Inertial Measurement Unit (IMU). The freaking sensor has on board a 16-bit ADC, i.e., an analogue-to-digital converter to convert physical acceleration on three axes (X, Y, Z). The I2C communication bus has a clock speed of 100 kHz and is used to establish communication between the MPU6050 and the ESP32. Automatic system intervention is actuated using a 5V relay module that interfaces with different digital GPIO pins, cutting off the motor power circuit during critical fault detection.

B. Deterministic Data Acquisition and Preprocessing

The predictive model is trained on a supervised data set that records the physical responses of the motor under four different operating conditions, namely, off (ambient environmental noise), normal (ataxia mode healthy operation), shaft fault (simulated rotor unbalance/misalignment), and mounting fault (simulated base looseness).

Temporal Precision: The data acquisition is controlled by a non-blocking microsecond timer that limits the sampling time interval (T_s) to exactly 10,000 μs. This results in an accurate sampling frequency (f_s) of 100 Hz that fully complies with the Nyquist-Shannon sampling theorem for mechanical macro-faults of interest in this study. As we know, MEMS sensors are prone to constant baseline drift due to thermal expansion resulting from the heat of nearby motors. In order to mitigate thermal drift, the "Normal" dataset consisted of long, continuous recordings (over 6 min). As a result, the neural network had to non-linearly characterise the physical baseline shift as a normal operating condition rather than a structural fault.

Non-linear Algorithmic Filtering: Before buffering, the digitised 3-axis readings are processed through a custom non-linear software filter. This filter removes high-frequency, transient mechanical spikes. Let $x(t)$ denote the raw sensor reading at time t, and $X(t-1)$ denote the previously accepted filtered value. The filtered output is determined by a spike limit ($\Delta_{\text{limit}} = 0.20\,\text{G}$) as follows:

$$y(t) = \begin{cases} x(t) & \text{if } |x(t) - X(t-1)| < \Delta_{\text{limit}} \\ x(t-1) & \text{otherwise} \end{cases} \tag{1}$$

C. Tiny ML Pipeline and Neural Network Optimisation

The Edge Impulse platform was used to design the predictive model, which uses raw time series kinematics as an input for an embedded classifier.

Digital Signal Processing (Spectral Analysis): Shallow neural networks find it computationally difficult to classify time-domain vibration signatures directly. As a result, a DSP block applying Fast Fourier Transform (FFT) processes the 600-element input tensor. The FFT enables entry of mere motion due to a fault. A shaft fault's mechanical thumping equates to a low-frequency motion, while the high-frequency chaos due to a mounting fault creates chaos in the high-frequency region. This essentially allows for obtaining highly separable, non-overlapping spectral peaks.

Sequential Keras Classifier: This forms the input vector for a deep, fully connected feed-forward neural network. Activation functions for the hidden layers are rectified linear units. The model underwent training across 100 epochs with a learning rate of 0.005. The model has used Data Augmentation (randomised Gaussian noise insertion) to mitigate overfitting.

Probabilistic Output (Softmax): The output layer has four nodes, which depict operational states. A softmax activation function transforms logits into a normalized probability distribution in the range $[0.0, 1.0]$.

Post-Training Quantisation: To make our model compatible with the ESP32's limited SRAM, the floating-point 32-bit (`float32`) model underwent aggressive quantisation. Network weights and biases were mapped down to an 8-bit integer (`int8`) format. This compression exponentially reduced the flash memory footprint and accelerated inferencing latency to the millisecond scale, with negligible degradation to the F1 accuracy score.

D. Dual-Core Asymmetric Multiprocessing (FreeRTOS)

Embedded systems often fail in real-time because synchronous routine sensor reading gets blocked by asynchronous high-latency tasks like Wi-Fi handshake. The use of FreeRTOS (Real-Time Operating System) achieves asymmetric multiprocessing on the two physical cores of the ESP32, without the traditional single-threaded loops.

- **Core 1 (High-Priority Real-Time Task):** Assigned exclusively to deterministic execution. It handles the 100 Hz I^2C polling, applies the algorithmic filter, manages the sliding array buffer, executes the quantised neural network (`run_classifier`), and enforces the local fault-actuation logic.
- **Core 0 (Low-Priority Asynchronous Task):** Dedicated solely to the networking stack. A pinned task (`Task_Cloud`) is spawned during the boot sequence, establishing the 802.11 Wi-Fi connection and lingering in a continuous loop to monitor outbound cloud traffic.

1) Inter-Core Communication via Queue (FreeRTOS): For exchanging data between the processes running on isolated processing cores while preventing race conditions and memory access violations, a thread-safe FreeRTOS Queue is used as the main IPC channel.

After an inference cycle, Core 1 generates a `Cloud_Packet` payload containing the peak-to-peak vibration amplitude, total fault strike count and an integer status code. Core 1 pushes this struct into the Queue, and immediately resuming acquisition of the sensor at 100 Hz. Core 0 continuously polls this Queue; when it finds a payload, Core 0 extracts the data and sends an HTTP packet. This separation guarantees that the neural network will not lose a mechanical frame during network unavailability.

E. Heuristic Fault Isolation and Majority Voting Logic

Shutting down an industrial machine based on a single erroneous inference from a neural network could lead to costly mistakes. To guarantee complete operational dependability, a multi-layered heuristic decision matrix filters the Softmax probability outputs of the AI engine.

Physical Amplitude Gate: Before performing inference, the absolute amplitude range of the windowed buffer is computed as:

$$A_{\text{score}} = \max(x) - \min(x) \tag{2}$$

If A_{score} is below the noise threshold of 0.05 G, then the software will bypass the neural network by overriding the status to *Off* and avoid any erratic predictions at low signals.

Confidence Floor: If the peak probability output from the Softmax layer is ≤ 0.50, the signal will be designated as ambiguous, and the system will default to the normal baseline state.

Temporal Majority Voting Algorithm: The methodology has a 3-strike fault tolerance with counting separate independent classes C_{shaft} and C_{mount}. If the probability of fault exceeds 0.50, the respective counter is incremented. The relay shut-off is triggered only when both fault counters reach a critical value, that is:

$$C_{\text{shaft}} + C_{\text{mount}} \geq 3 \tag{3}$$

Thus, both counters' sum must be at least three to trigger a relay shutdown.

Fault Arbitration: After the relay shutdown is triggered, the highest integer within the strike array is arbitrated. For instance, if $C_{\text{shaft}} = 2$ and $C_{\text{mount}} = 1$, the final output will be *Shaft Fault*. This guarantees that the most persistent time-dependent fault governs the maintenance response.

F. IoT Telemetry and Application Layer Protocol

The system functions as a robust IoT edge computing node. Cloud telemetry is routed to ThingSpeak, an IoT analytics aggregator. The Core 0 `Task_Cloud` utilises the REST (Representational State Transfer) API architecture, dispatching data via HTTP POST requests.

The `millis()` function is used to cause a non-blocking time delay in accordance with the rate limiting on the cloud server. Hence, sending is done with a maximum rate of once every 15 seconds. The transmission payload passes the extracted vibration strength, the accumulation of the active fault counter, and the current system status code into different cloud channels. This lets supervisors off-site keep a tab on asset degradation in real-time on web-based dashboards before local automatic shutdowns.

G. System Integration and Operational Workflow

On power-up, the ESP32 initialises both cores under FreeRTOS simultaneously. Core 0 establishes Wi-Fi and launches `Task_Cloud`, while Core 1 calibrates the MPU6050 and energises the relay to start the motor. A deliberate 3-second startup delay is enforced to let the motor settle, preventing the initial mechanical jerk from corrupting the first inference window.

Once active, the system runs a continuous non-blocking loop, sampling every 10 ms. When the 600-sample buffer fills, the full pipeline fires on Core 1: amplitude gate, inference, confidence check, and relay arbitration, while Core 0 independently handles cloud uploads without disturbing the sensing rhythm.

This makes the proposed system particularly attractive for unmanned or remote industrial installations, where human intervention is either delayed or impractical. Unlike traditional threshold-based alarms that fire on a single anomalous reading, this node cross-validates every decision across time, amplitude, and confidence before acting. The result is a system that is neither too sensitive to trip on noise, nor too sluggish to miss a developing fault.

IV. Flowchart of the Code

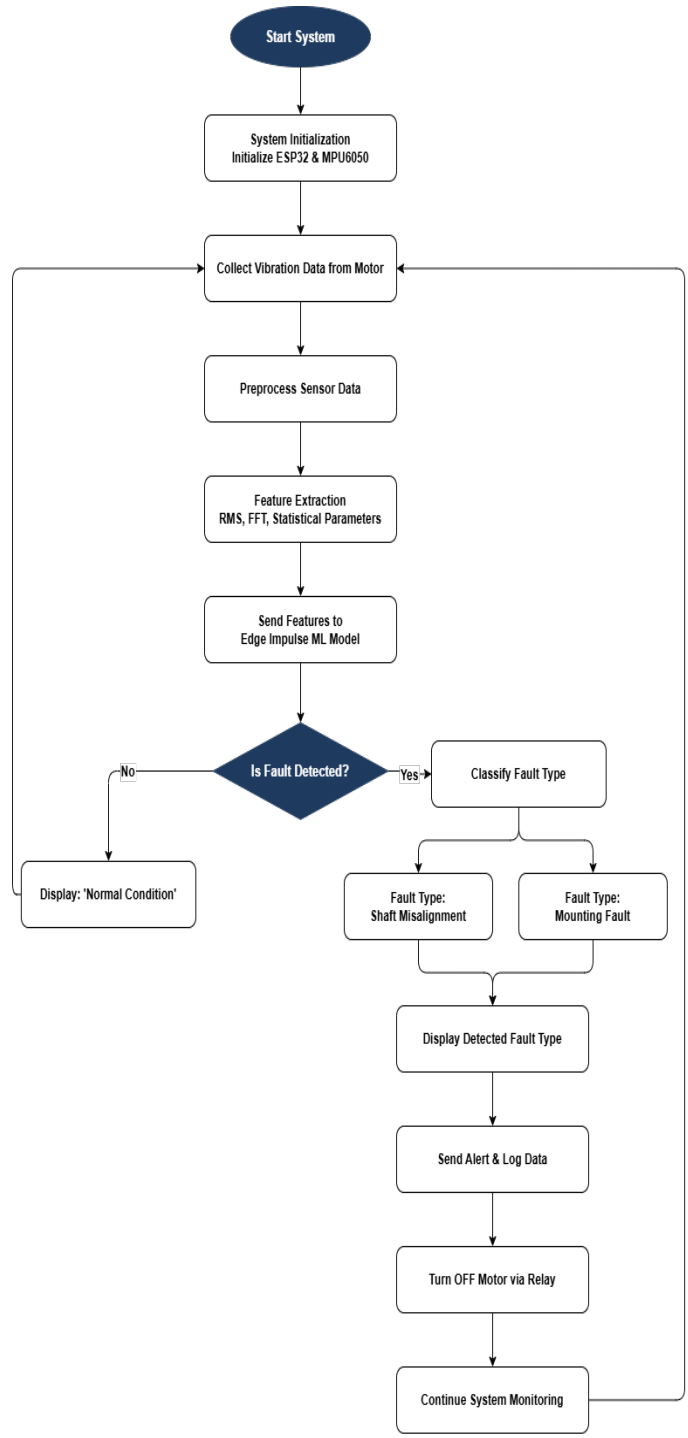

Fig. 1. Flowchart for Fault Detection Algorithm

V. Results and Discussion

A. Simulation Results

The predictive maintenance model was first verified using Edge Impulse simulation tools. The trained model was evaluated using the confusion matrix, classification accuracy, and feature extraction visualization.

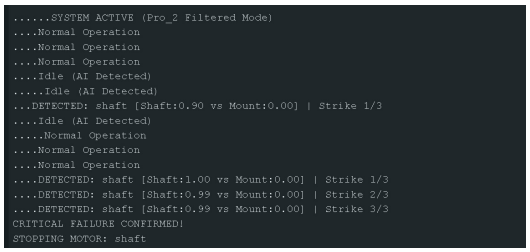

Fig. 2. Shaft Fault Detection

The serial monitor output in Fig. 2 shows the detection of a shaft misalignment fault. The system first operates under normal conditions and then goes to a fault state. Once the classifier detects shaft fault signatures in three consecutive windows (Strike 1/3, 2/3, 3/3), the majority vote logic confirms that there is a fault. Upon confirmation, the system prints Shaft Fault or Mounting Fault and immediately cuts power to the motor through the relay, preventing further damage.

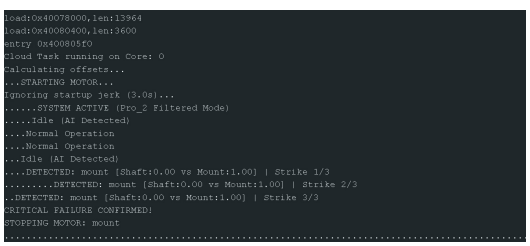

Fig. 3. Mounting Fault Detection

Fig. 3 shows the system response during a mounting fault condition. The MPU6050 sensor detects abnormal vibration patterns caused by improper motor mounting. The Edge Impulse model classifies the vibrational patterns as a mounting fault, and the strike counter increments. After three confirmed detections, the system sends a critical failure alert and switches off the motor via the relay output, ensuring safe operation.

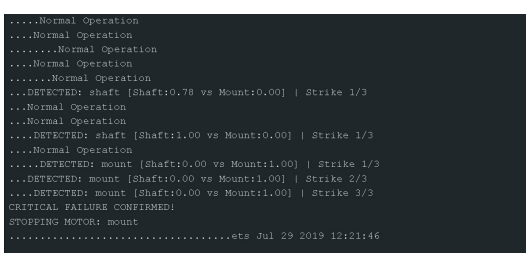

Fig. 4. Mixed Detection and System Response

Fig. 4 shows the condition where the system encounters both shaft and mounting fault patterns during the same operating session. The classifier gives probability scores for both fault classes in each window, and the dominant fault type is determined using a majority logic in the code itself. The system correctly identifies the primary fault and responds by stopping the motor, demonstrating the accuracy of the code in the overlapping condition.

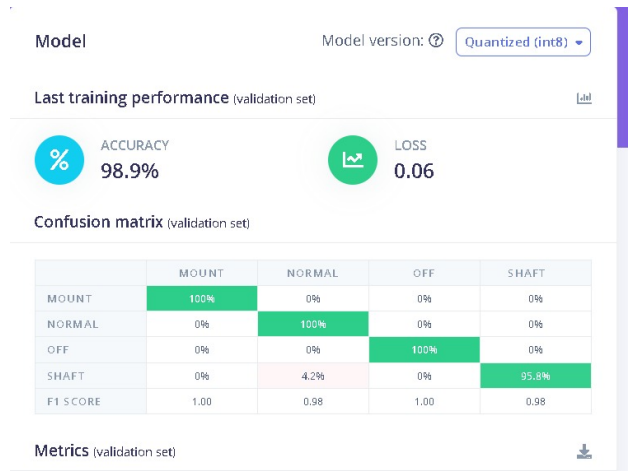

Fig. 5. Model Accuracy

Fig. 6. Graph Representation

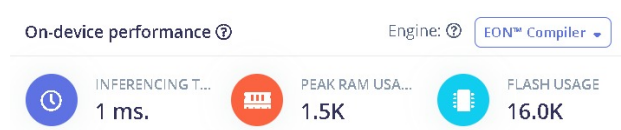

Fig. 7. Device Performance

Feature visualization confirms that vibration signatures for different motor conditions are distinguishable, enabling accurate classification using the trained Edge AI model.

B. Model Training and Classification Details

Quantized (int8) neural network classifier was used to train and verify the data on Edge Impulse Platform.Basically four classes were there-idle,normal,shaft fault and mount fault.

For training,a total of 5800 samples were used.The trained model achieved a classification accuracy of 98.9% with a validation loss of 0.06.As clearly seen from the confusion matrix, the model correctly identified all classes when present. For shaft fault detection, the model achieved 95.8% accuracy.The F1 scores for mount, normal, off, and shaft classes were 1.00, 0.98, 1.00, and 0.98 respectively, confirming strong and balanced classification performance across all fault types.

Through Fig.7 one can observe that the vibrational patterns for all four motor conditions are well spaced in the graphical representation, which confirms that the model is actually able to classify different working and failing conditions, providing probability for each to occur.

C. Hardware Implementation

The complete hardware setup was implemented on a breadboard using ESP32, MPU6050 sensor, and a 12V DC motor. The vibration sensor was placed on top circular surface of the 12V Motor, for getting the best vibration pattern readings.

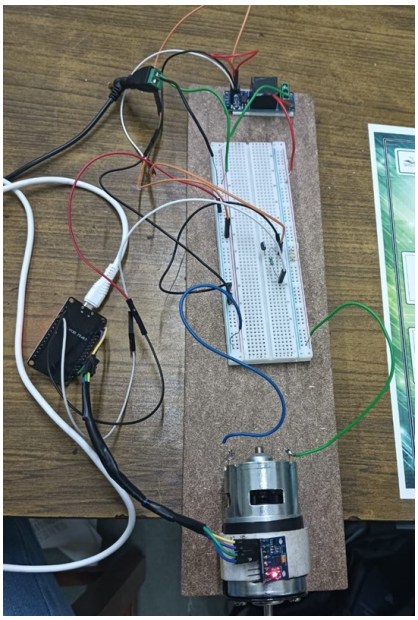

Fig. 8. Actual Hardware Setup of IoT Predictive Maintenance Node

The trained Edge Impulse model was integrated into the ESP32 microcontroller. The system classified motor conditions in real-time and displayed results through serial monitor.

- Power supply: 12V for motor, 5V for ESP32
- Real-time Serial output of fault type
- Stable operation without internet connectivity

D. PCB Design and Development

To make sure that the project can be implemented easily in industries, the entire schematic containing the components was made and integrated into a Printed Circuit Board in KiCad 9.0 software. The main reason PCB is beneficial is that it avoids the condition of lose wires floating around, also brings all components into a single board which can be directly mounted on top of motor, no extra hardware needed.

1) Schematic Design: The schematic was designed using KiCad 9.0. The design includes:

- ESP32-WROOM module
- MPU6050 sensor interface
- BJT BC547
- Relay
- Resistor
- Header Pins for Male and Female Wires

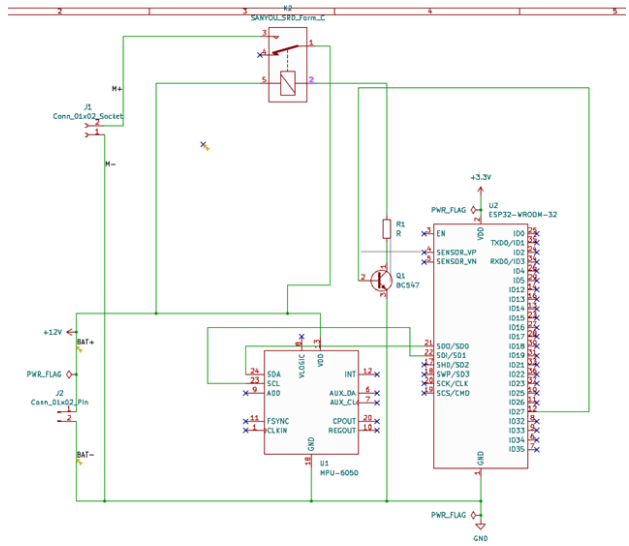

Fig. 9. Schematic Layout of the IoT Predictive Maintenance PCB

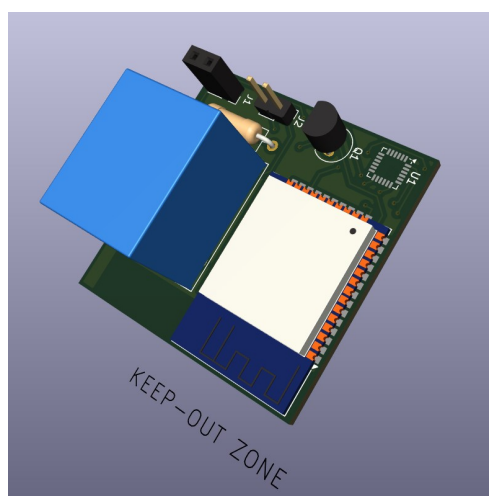

Fig. 10. Final PCB Layout for Industrial Deployment

2) PCB Layout Design: The PCB layout was made by keeping some factors in mind, factors like size, overall stability, and noise immunity.Some good steps taken for the design are as follows:

- Ground plane implementation
- Short signal routing for I2C lines

- Isolation between motor power and logic section
- Proper trace width for current handling

The final PCB can be directly mounted on the motor housing, making it suitable for real industrial applications.

VI. Conclusion

This paper baically presented how Internet of Things is used and incorporated into a compact intelligent predicitive maintenance node for determining type of fault in industrial motors. The system uses the ESP32 microcontroller, MPU6050 vibration sensor, and 12V direct current motor to obtain accurate real-time vibration data and classify various faults of the motor, like shaft misalignment and mounting issues.

The system proposed is different from current systems in a way that it uses AI model which is directly embedded into the chip. This reduces latency, bandwidth, and dependence on the internet. Integrating machine learning model indicates that any Edge AI model can be easily put into a microcontroller system, to avoid waiting for cloud based operations.

The proposed system was made from scratch, starting from a basic breadboard implementation to a fully developed industrial-grade printed circuit board (PCB) module software design. This improved the overall reliability of the system and enabled it to be directly attached to an industrial motor. Perfect operation was shown by the hardware implementation. The system was able to detect faults in real-time with high accuracy.

In general, the proposed system presents a cost-effective, scalable, and industrially applicable predictive maintenance solution for Industry 4.0. For future addition and scope, faults like bearing wear can also be detected using some mathematical relations of vibrational patterns. The system can be extended to a cloud-based monitoring solution for a range of machine monitoring applications.

References

[1] S. Yin and O. Kaynak, "Big Data for Modern Industry," *Proceedings of the IEEE*, vol. 103, no. 2, pp. 143–146, 2015.

[2] J. Wan et al., "Software-Defined Industrial Internet of Things," *IEEE Sensors Journal*, vol. 16, no. 20, pp. 7373–7380, 2016.

[3] H. Li, K. Ota, and M. Dong, "Learning IoT in Edge," *IEEE Network*, vol. 32, no. 1, pp. 96–101, 2018.

[4] Z. Zhou et al., "Edge Intelligence," *Proceedings of the IEEE*, vol. 107, no. 8, pp. 1738–1762, 2019.

[5] H. Uzel et al., "Acoustic-Based Fault Diagnosis of Electric Motors," *Scientific Reports*, 2025.

[6] S. Ullah et al., "Spatio-Temporal Feature Extraction for Pipeline Leak Detection," *Applied Sciences*, 2024.

[7] Y. Lei et al., "Applications of Machine Learning to Machine Fault Diagnosis," *Mechanical Systems and Signal Processing*, vol. 138, 2020.

[8] W. Zhang et al., "A New Deep Learning Model for Fault Diagnosis," *Sensors*, vol. 17, no. 2, p. 425, 2017.

[9] P. Warden and D. Situnayake, *TinyML*. O'Reilly Media, 2019.

[10] C. Banbury et al., "MLPerf Tiny Benchmark," *arXiv preprint arXiv:2106.07597*, 2021.

[11] Espressif Systems, "ESP32 Technical Reference Manual," 2023.

[12] InvenSense Inc., "MPU-6050 Product Specification," Datasheet, 2013.

[13] Edge Impulse Inc., "Edge Impulse Documentation," 2023.

Appendix A
ESP32 Edge AI Code (Simplified)

```
#include <pro_3_inferencing.h>
#include <Wire.h>
#include <MPU6050_light.h>
#include <WiFi.h>
#include <ThingSpeak.h>

const char* ssid = "Tejas";
const char* password = "123456789";

unsigned long myChannelNumber = 3239801;
const char * myWriteAPIKey = "GEWA727F6FESXTCR";

WiFiClient client;
MPU6050 mpu(Wire);
const int relay = 12;

float SPIKE_LIMIT = 0.20;
float NOISE_THRESHOLD = 0.05;

const unsigned long INTERVAL_US = 10000;
unsigned long lastSampleTime = 0;
#define EI_WINDOW_SIZE (EI_CLASSIFIER_DSP_INPUT_FRAME_SIZE)
float features[EI_WINDOW_SIZE];
int featureIndex = 0;

int shaftFaultCount = 0;
int mountFaultCount = 0;

float lastX = 0, lastY = 0, lastZ = 1.0;
float minX = 100, maxX = -100;

struct CloudPacket {
  float vibration;
  int faultCount;
  int statusCode;
};
QueueHandle_t cloudQueue;

void TaskCloud(void *pvParameters) {
  Serial.print("Cloud Task running on Core: ");
  Serial.println(xPortGetCoreID());

  WiFi.mode(WIFI_STA);
  WiFi.begin(ssid, password);

  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }

  Serial.println("\nWiFi Connected!");
  Serial.println(WiFi.localIP());

  ThingSpeak.begin(client);
  CloudPacket packet;
  unsigned long lastUploadTime = 0;

  for (;;) {
    if (xQueueReceive(cloudQueue, &packet, portMAX_DELAY) ==
      pdPASS) {
      if (millis() - lastUploadTime > 15000) {
        Serial.println("Uploading to ThingSpeak...");
        ThingSpeak.setField(1, packet.vibration);
        ThingSpeak.setField(2, packet.faultCount);
        ThingSpeak.setField(3, packet.statusCode);
        int x = ThingSpeak.writeFields(myChannelNumber,
     myWriteAPIKey);
        if(x == 200) Serial.println("Upload Success");
        lastUploadTime = millis();
      }
    }
  }
}

void setup() {
  pinMode(relay, OUTPUT);
  Serial.begin(115200);

  cloudQueue = xQueueCreate(5, sizeof(CloudPacket));
```

```
  xTaskCreatePinnedToCore(TaskCloud, "CloudTask", 10000,
     NULL, 1, NULL, 0);

  Wire.begin(21, 22);
  Wire.setClock(100000);
  byte status = mpu.begin();
  if(status != 0){
    Serial.println("Sensor Missing!");
    while(1){ delay(10); }
  }

  Serial.println("Calculating offsets...");
  delay(1000);
  mpu.calcOffsets();

  Serial.println("STARTING MOTOR...");
  digitalWrite(relay, HIGH);

  Serial.println("Ignoring startup jerk (3.0s)...");
  delay(3000);

  Serial.println("SYSTEM ACTIVE (Majority Vote Mode)");
  lastSampleTime = micros();
}

void loop() {
  if (micros() - lastSampleTime >= INTERVAL_US) {
    lastSampleTime += INTERVAL_US;
    mpu.update();

    float rawX = mpu.getAccX();
    float rawY = mpu.getAccY();
    float rawZ = mpu.getAccZ();

    if (abs(rawX - lastX) < SPIKE_LIMIT) lastX = rawX;
    if (abs(rawY - lastY) < SPIKE_LIMIT) lastY = rawY;
    if (abs(rawZ - lastZ) < SPIKE_LIMIT) lastZ = rawZ;

    if (featureIndex < EI_WINDOW_SIZE) {
        features[featureIndex++] = lastX;
        features[featureIndex++] = lastY;
        features[featureIndex++] = lastZ;

        if (lastX < minX) minX = lastX;
        if (lastX > maxX) maxX = lastX;
    }
    else {
        float vibrationStrength = maxX - minX;
        int currentStatus = 0;
        int totalStrikes = shaftFaultCount + mountFaultCount
     ;

        if (vibrationStrength < NOISE_THRESHOLD) {
             if (totalStrikes > 0) Serial.println("Counters
     Reset.");
             shaftFaultCount = 0;
             mountFaultCount = 0;
             currentStatus = 0;
             Serial.println("Idle (Low Vibration)");
        }
        else {
            signal_t signal;
            numpy::signal_from_buffer(features,
     EI_WINDOW_SIZE, &signal);
            ei_impulse_result_t result;
            run_classifier(&signal, &result, false);

            String topLabel = "Unknown";
            float maxConfidence = 0.0;

            float shaftScore = 0.0;
            float mountScore = 0.0;

            for (uint16_t i = 0; i <
     EI_CLASSIFIER_LABEL_COUNT; i++) {
                String label = result.classification[i].
     label;
                float value = result.classification[i].value
     ;

                if (label == "shaft") shaftScore = value;
                if (label == "mount") mountScore = value;
                if (value > maxConfidence) {
                    maxConfidence = value;
                    topLabel = label;
                }
            }

            String mainStatus;

            if (maxConfidence > 0.50) {
                mainStatus = topLabel;
            } else {
                mainStatus = "normal";
            }

            if (mainStatus == "shaft" || mainStatus == "
     mount") {
                if (mainStatus == "shaft") shaftFaultCount
     ++;
                if (mainStatus == "mount") mountFaultCount
     ++;

                totalStrikes = shaftFaultCount +
     mountFaultCount;
                currentStatus = 2;

                Serial.print("DETECTED: "); Serial.print(
     mainStatus);
                Serial.printf(" [Shaft:%.2f vs Mount:%.2f] |
      Strike %d/3\n", shaftScore, mountScore, totalStrikes);

                if (totalStrikes >= 3) {
                    String dominantFault = (shaftFaultCount
     > mountFaultCount) ? "shaft" : "mount";

                    Serial.println("CRITICAL FAILURE
     CONFIRMED!");
                    Serial.print("STOPPING MOTOR DUE TO
     MAJORITY FAULT: "); Serial.println(dominantFault);

                    digitalWrite(relay, LOW);

                    CloudPacket deathMsg;
                    deathMsg.vibration=vibrationStrength;
                    deathMsg.faultCount=3;
                    deathMsg.statusCode=2;
                    xQueueSend(cloudQueue, &deathMsg, 0);

                    while(1) { delay(100); }
                }
            }
            else if (mainStatus == "normal") {
                 shaftFaultCount = 0;
                 mountFaultCount = 0;
                 currentStatus = 1;
                 Serial.println("Normal Operation");
            }
            else if (mainStatus == "off") {
                 shaftFaultCount = 0;
                 mountFaultCount = 0;
                 currentStatus = 0;
                 Serial.println("Idle (AI Detected)");
            }
        }

        CloudPacket msg;
        msg.vibration = vibrationStrength;
        msg.faultCount = shaftFaultCount + mountFaultCount;
        msg.statusCode = currentStatus;
        xQueueSend(cloudQueue, &msg, 0);

        featureIndex = 0;
        minX = 100; maxX = -100;
    }
  }
}
```